07 · Problem 2: The Extra Step Adds Delay

Step 1: What happens

After the LLM finishes writing the query, the system has to wait while the embedding model processes that text from scratch. Only then can the index be searched.

Step 2: Why it hurts

Agents may search many times per conversation, and each search sits directly on the path to the answer. The delays pile up:

Searches in one agent task	Extra delay at 43.5 ms each
1	0.04 s
10	0.44 s
50	2.2 s

For a single chatbot reply this is small. For multi-step agents, or for a service handling thousands of users, it adds up to real waiting time and real GPU load.

Step 3: A concrete example (the paper's numbers)

Measured on one NVIDIA RTX PRO 6000:

Stage	Baseline	Projection head
Turning the query into a vector (p50)	**43.5 ms** (a full 8B embedder forward pass)	**2.0 ms** (a ~25M-parameter network over hidden states already in memory)

That is about 21.8× faster *for this step*. Note what this number does **not** include: both pipelines still spend time having the LLM generate the query, and both still search the index. The saving is in the encode step only.

Step 4: How people patched it

- Smaller or quantized embedders (faster, usually a little worse).
- Caching embeddings of repeated queries (helps only when queries repeat, which is rare in conversations).
- Running the embedder on the same GPU to skip network hops.

None of these remove the step.

Step 5: What a real fix would need

Produce the query vector from something that is **already computed**, using so little extra work that the step almost disappears.

→ Answered by solution steps `12` (keep the hidden states) and `13` (a tiny projection head).